FINAL PROJECT REPORT

Automated Polyglot Data Pipeline for Governance Data

Applied Data Science and Artificial Intelligence

Group 2 — The Data Orchestrators

Guna Prakash Kethineni | Kaustubh Zanan | Manav Chauhan | Niranjan V

February 20, 2026

1. Executive Summary & Architecture Overview

Public city council meeting records generate massive, unstructured text transcripts. Without a structured data pipeline, this "dark data" is incredibly difficult to query or analyze. Group 2 — "The Data Orchestrators" — successfully built a fully automated, polyglot data pipeline to process a representative subset of the MeetingBank dataset.

We designed a comprehensive 5-Layer Architecture System:

- Orchestration Layer: Managed entirely by Prefect, acting as the "brain" for DAG scheduling, retries, and monitoring.
- Ingestion & Processing Layer: Utilizing Python and Pandas to extract raw Excel data, clean it, and perform real-time feature engineering.
- Storage (Polyglot) Layer: Routing structured metadata to MySQL and massive unstructured transcripts to MongoDB.
- Monitoring & Alerting Layer: Leveraging Prefect's run_logger and event-driven on_completion / on_failure hooks for full pipeline visibility.
- Analytics & Visualization Layer: Utilizing SQLAlchemy to feed a live Streamlit dashboard.

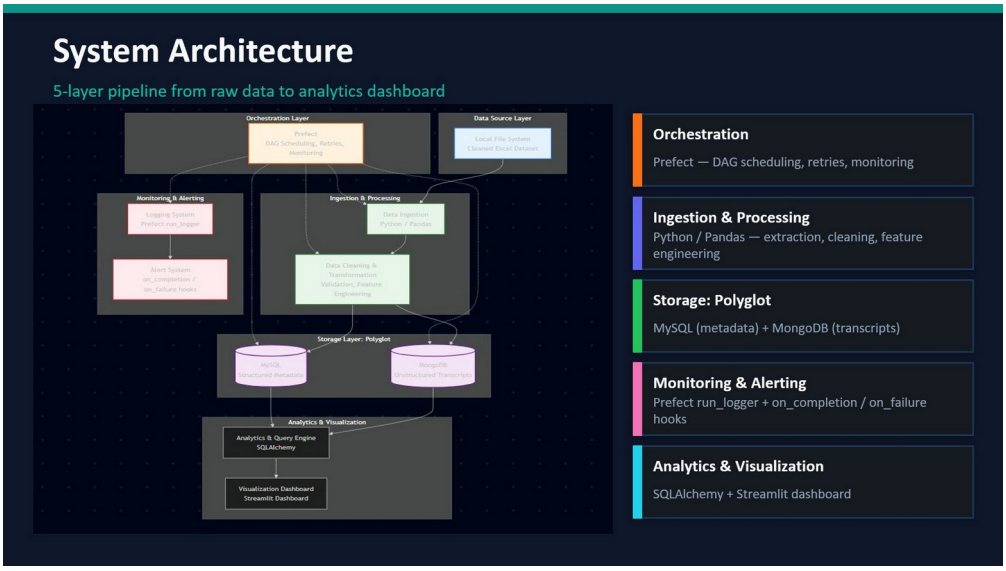

Figure 1: 5-Layer System Architecture Diagram — data flows from local ingestion through Prefect orchestration into Polyglot storage and out to the BI dashboard.

2. Core Task 1: Data Ingestion and Cleaning

Our first priority was ensuring that our database only ingested high-quality data. We utilized the pandas library to handle the raw dataset containing meeting transcripts, summaries, and metadata.

Instead of blindly loading the data, we programmed strict validation rules. We standardized all column headers to lowercase to prevent mapping errors and immediately dropped any corrupted rows that lacked a Unique ID (uid).

Code Snippet: Ingestion and Validation

```
@task(name="Ingest Data", retries=2, retry_delay_seconds=5)
def extract_data():
    # Load the raw Excel dataset
    df = pd.read_excel(EXCEL_PATH)

    # Clean and normalize raw data headers
    df.columns = df.columns.str.strip().str.lower()

    # Validation: Drop any rows missing a critical Unique ID
    df = df.dropna(subset=["uid"])
    return df
```

3. Core Task 2: Data Transformation and Modeling

Raw text data is difficult to analyze mathematically. To create a structured schema suitable for downstream querying, we applied feature engineering to derive new analytical attributes from the raw transcripts.

We wrote custom Lambda functions to dynamically calculate word_count (to measure meeting intensity) and char_length (used as a data quality validation metric). We also parsed the uid string to extract the specific city name for operational grouping.

Code Snippet: Feature Engineering

```
@task(name="Transform & Clean Data")
def transform_data(df):
    # Derive additional attributes: Word count and Character length
    df["word_count"] = df["transcript"].astype(str).apply(lambda x: len(x.split()))
    df["char_length"] = df["transcript"].astype(str).apply(len)

    # Extract the city name from the UID prefix for downstream grouping
    df["city"] = df["uid"].astype(str).apply(lambda x: x.split("_")[0].capitalize())
    return df
```

4. Core Task 3: Data Storage — Polyglot Architecture

We implemented a Polyglot Storage Architecture because a single database type cannot efficiently handle both rapid analytical queries and massive 10,000+ word unstructured text documents.

- Relational Database (MySQL): Stored structured metadata — id, uid, summary, word_count, char_length, loaded_at — enabling lightning-fast aggregations.
- NoSQL Database (MongoDB): Stored heavy, unstructured meeting transcripts mapped to their uid as JSON documents. MongoDB's schema-less design prevents row-size truncation and text corruption.

To verify our relational database design, we executed SQL queries directly in MySQL Workbench:

```
SELECT * FROM meeting_pipeline.meetings_metadata LIMIT 0, 1000;
```

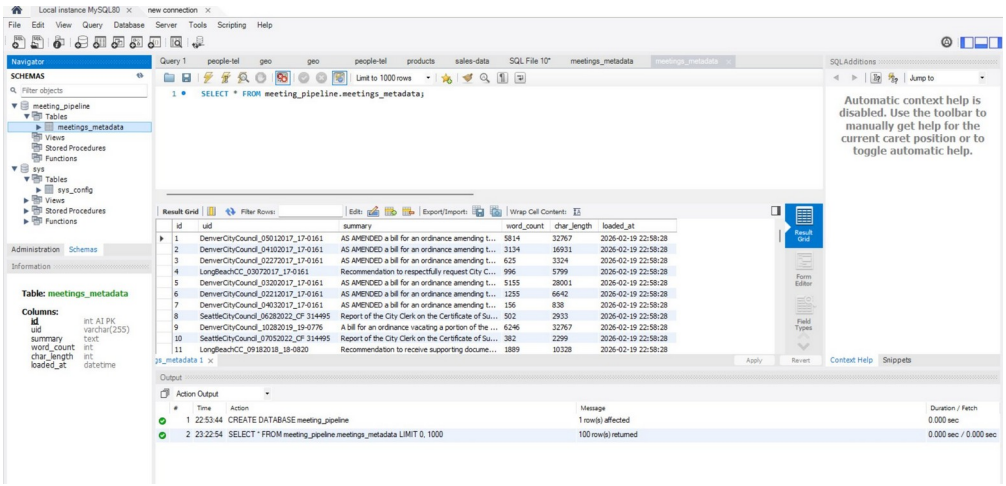

Figure 2: MySQL Workbench confirming successful creation of the structured meetings_metadata schema and 100-row data population.

5. Group Extension Task: Automation, Scheduling & Orchestration

As Group 2, our unique extension task was to implement Pipeline Orchestration, Automation, and Scheduling. We moved beyond manual scripts and utilized Prefect to build a self-healing, production-ready system.

5.1 Prefect Dashboard — Pipeline Overview

The Prefect Dashboard provides a live summary of all flow runs and task runs. It shows 46 total task runs with 34 completed (77.27%) and a clear trend of improving stability over time as retries resolved connection failures.

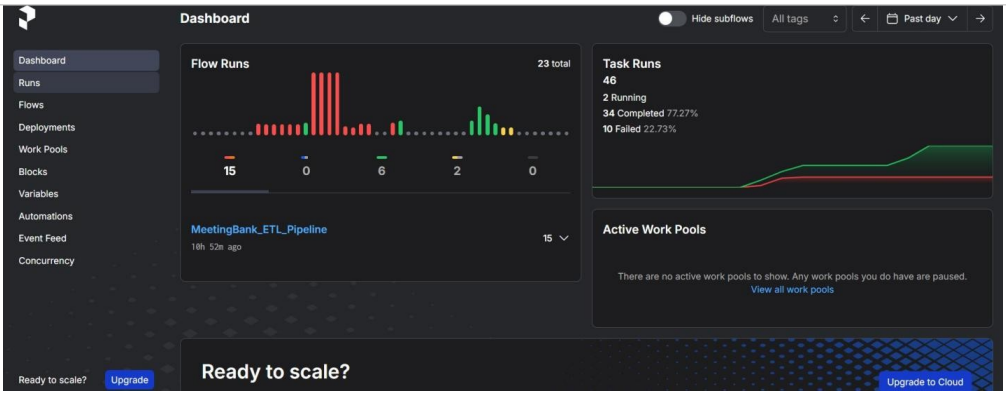

Figure 3: Prefect Dashboard showing 23 flow runs and 46 task runs for the MeetingBank_ETL_Pipeline, with a 77.27% task completion rate.

5.2 Directed Acyclic Graph (DAG) & Task Dependencies

We wrapped our Python functions in Prefect @task and @flow decorators, mapping strict execution rules. Our final analytics_summary task uses a wait_for flag, securely locking it until both load_mysql and load_mongodb report 100% successful data loads.

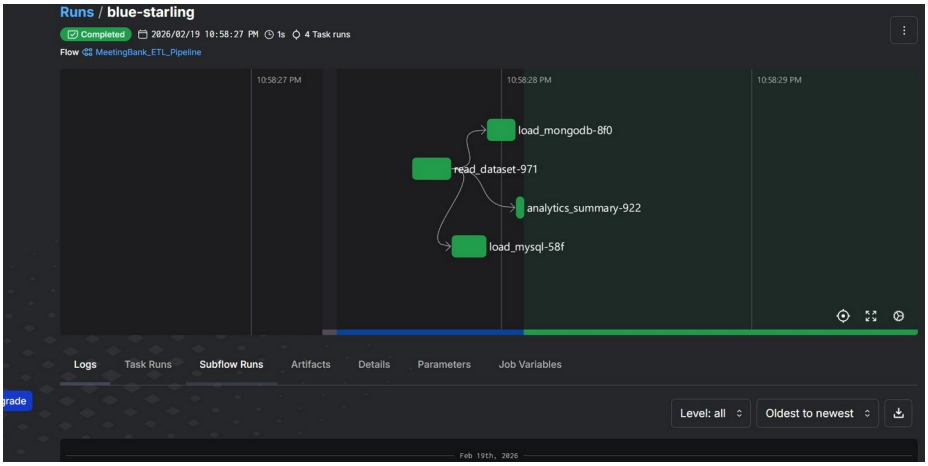

Figure 4: Prefect DAG execution graph for the "blue-starling" run — showing concurrent load_mysql and load_mongodb with analytics_summary locked downstream.

The detailed Prefect logs for the blue-starling run confirm the full execution sequence: dataset read (100 records), MySQL load completed, MongoDB load completed, and flow finished successfully.

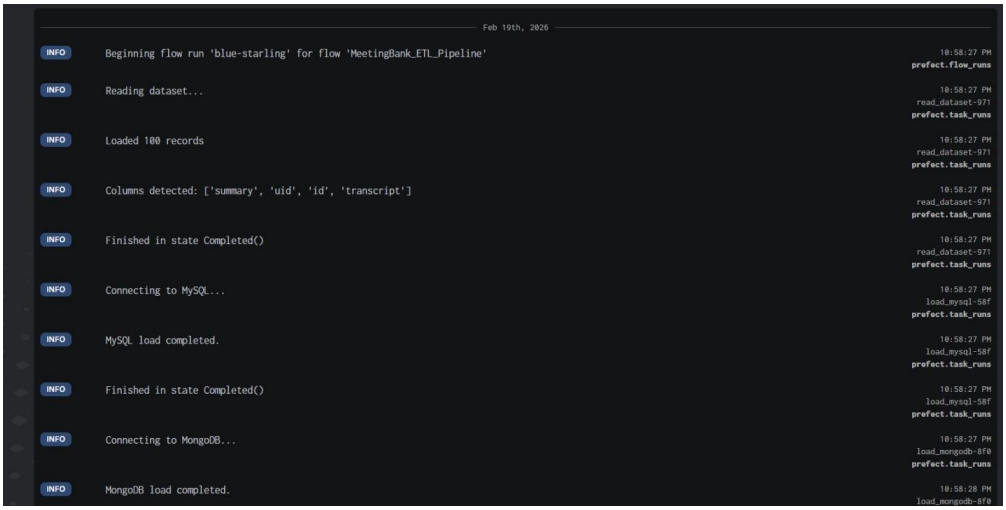

Figure 5: Prefect execution logs for the blue-starling run — confirming 100 records loaded, MySQL completed, MongoDB completed.

5.3 Run History and Fault Tolerance

The Prefect Runs page shows the complete scatter plot of all 24 flow runs over the past week, with red dots indicating failures (early connection drops) and green dots confirming successful self-healing retries.

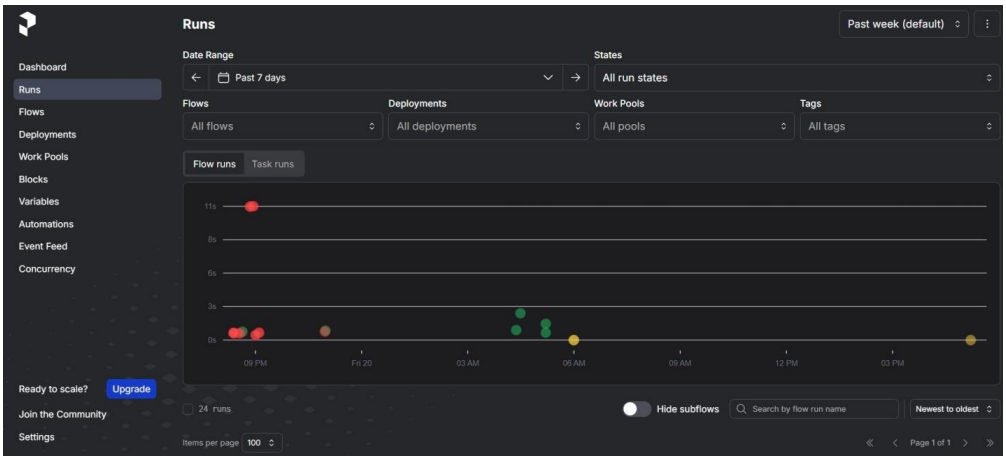

Figure 6: Prefect Runs scatter plot — red dots show failed runs (connection drops), green dots confirm successful self-healed retries.

The run list view clearly shows automated scheduling in action, with Completed runs alongside Late runs from the Bi-Daily-Backup and Daily-Morning-Sync deployments waiting for a worker.

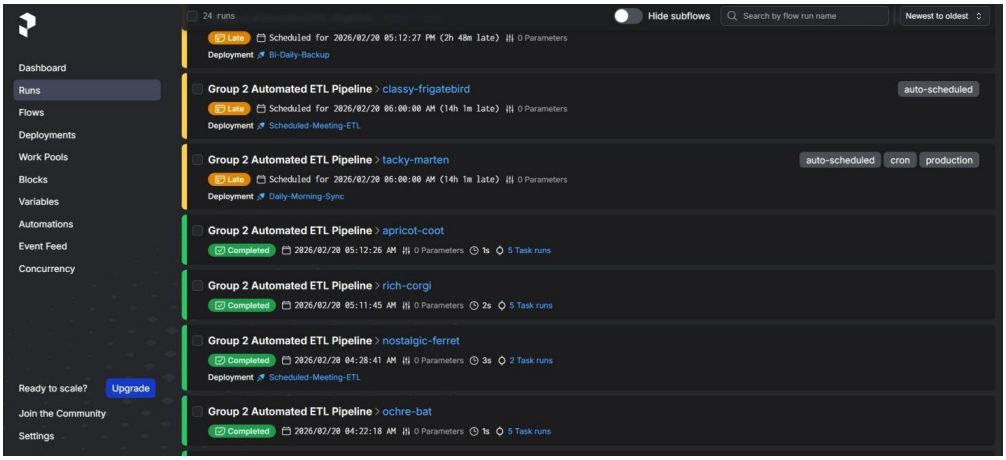

Figure 7: Prefect run list showing Completed runs (green) alongside Late auto-scheduled runs from CRON and interval deployments.

5.4 Flows Registry

Our Prefect instance registered two distinct flows: the original MeetingBank_ETL_Pipeline (last run: blue-starling) and the production Group 2 Automated ETL Pipeline with 3 active deployments.

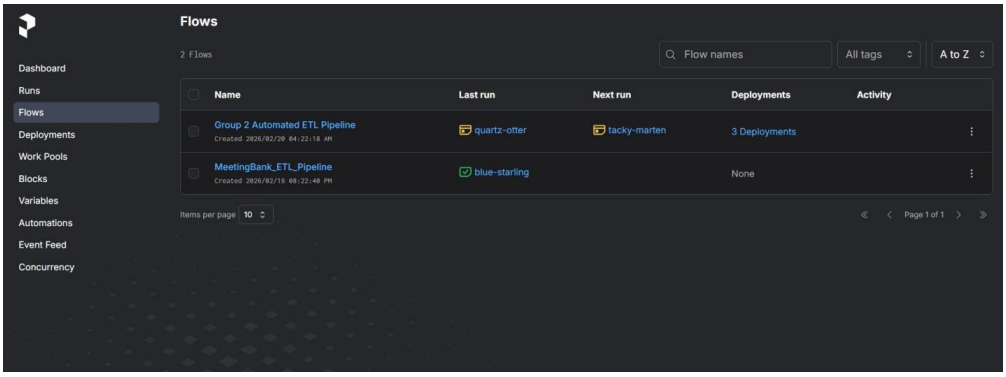

Figure 8: Prefect Flows page showing both registered flows — MeetingBank_ETL_Pipeline and the Group 2 Automated ETL Pipeline with 3 deployments.

5.5 Automated Scheduling (CRON & Interval Deployments)

We eliminated human intervention by deploying three automated workers: Daily-Morning-Sync (CRON at 05:00 AM every day), Bi-Daily-Backup (every 12 hours), and Scheduled-Meeting-ETL (at 05:00 AM every day).

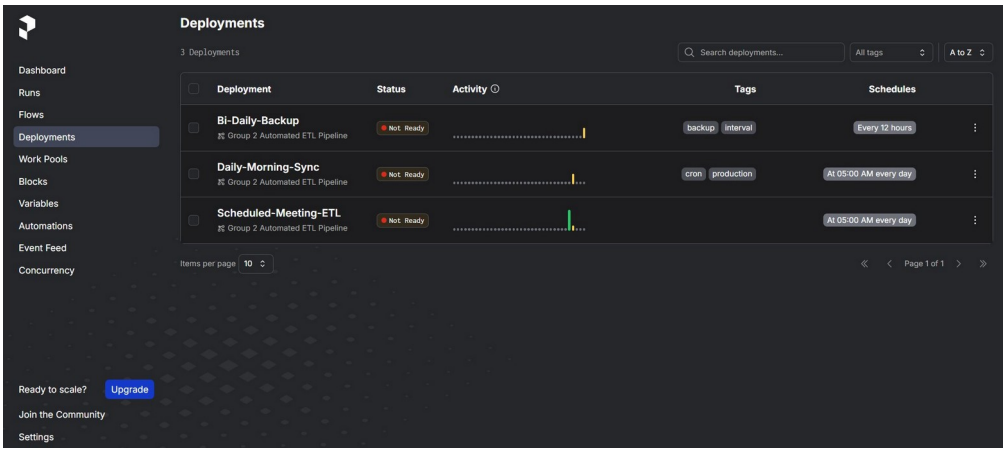

Figure 9: Prefect Deployments page showing all three configured schedules — CRON daily at 05:00 AM, and an interval-based 12-hour backup.

Drilling into the Bi-Daily-Backup deployment confirms the 12-hour interval schedule is active, the next run is already queued (splendid-goshawk), and the entrypoint file is correctly configured.

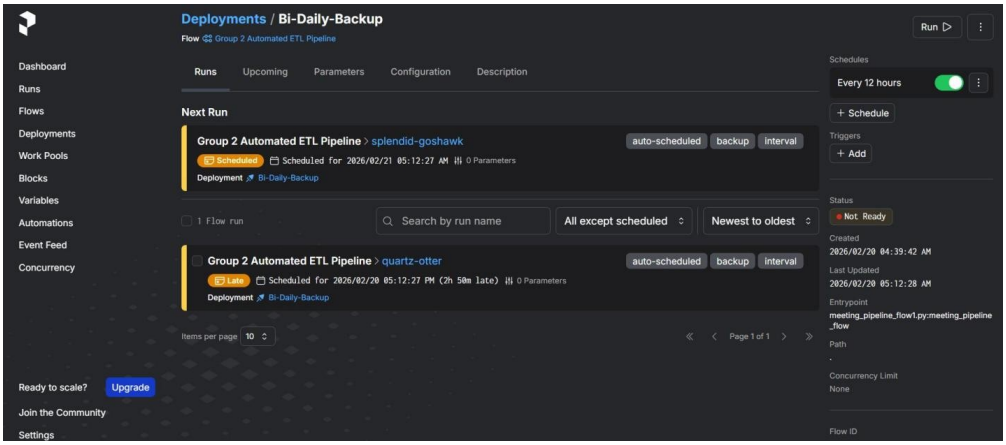

Figure 10: Bi-Daily-Backup deployment detail — confirming the "Every 12 hours" interval schedule, next run queued, and entrypoint configuration.

5.6 Event Feed — Automated Audit Trail

Prefect's Event Feed logs every state change as a timestamped, searchable audit trail. Both flow-level completions and individual task-level completions (such as the Run Analytics Query-feb task) are captured.

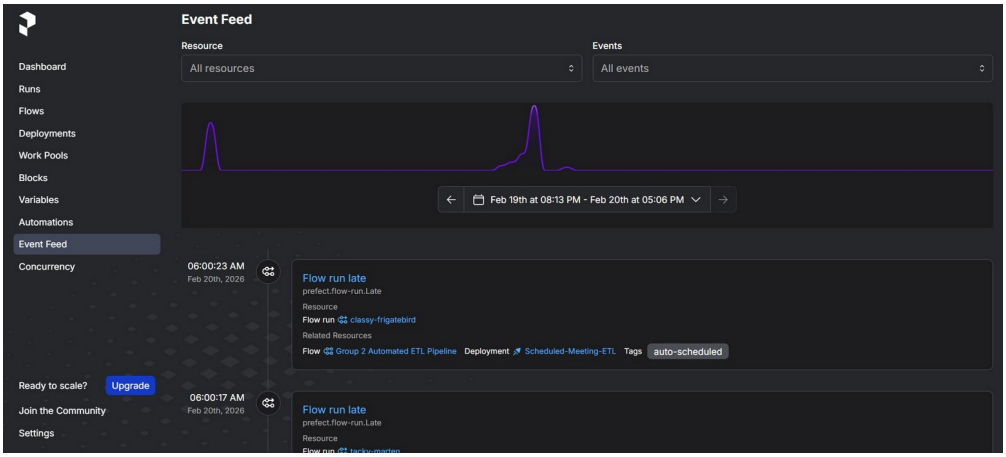

Figure 11: Prefect Event Feed overview — the purple activity graph and timestamped event log showing flow run late events from Feb 19–20.

Scrolling through the event feed reveals the complete lifecycle: late run alerts fire at the scheduled time, providing immediate visibility when a worker is unavailable to pick up a scheduled run.

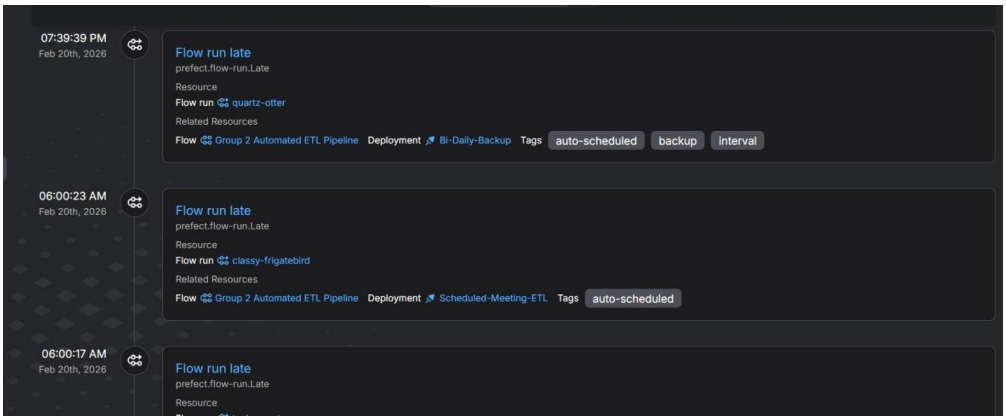

Figure 12: Event Feed scroll — late run events for Bi-Daily-Backup (quartz-otter) and Scheduled-Meeting-ETL (classy-frigatebird) captured precisely at their scheduled times.

When runs do complete successfully, the event feed captures both the flow run completed and task run completed events, providing a full end-to-end audit chain for every execution.

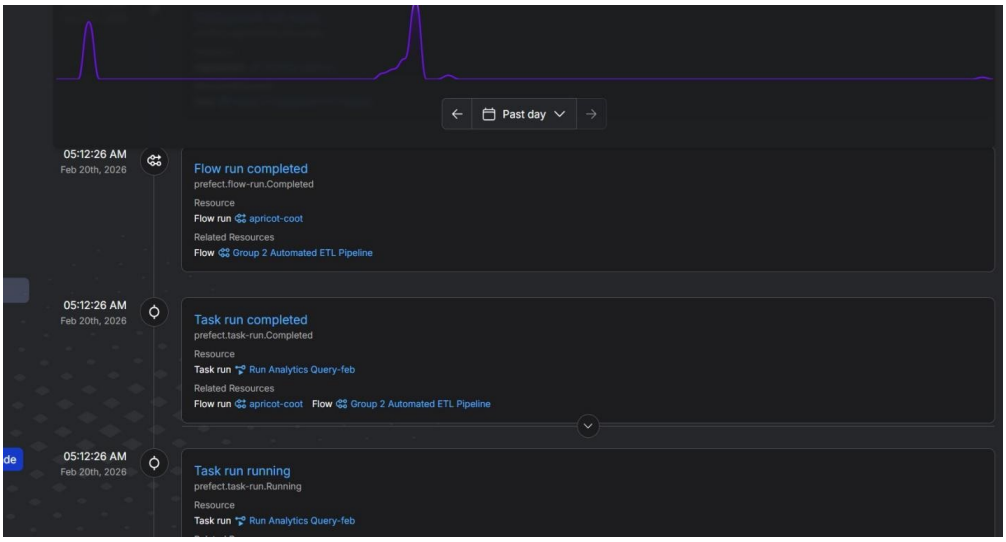

Figure 13: Event Feed showing "Flow run completed" and "Task run completed" events for the apricot-coot run, proving full audit chain coverage.

6. Core Task 4: Querying and Analysis

To demonstrate the usability of our data model, we connected a Streamlit BI dashboard directly to our MySQL instance using SQLAlchemy. All four analytics visualizations below are driven by live SQL queries against the meetings_metadata table.

6.1 Pipeline KPI Summary

High-level KPIs confirmed at a glance:

Metric	Value
Total Meetings Processed	100
Distinct Cities	6
Average Transcript Length	2,031 words
Longest Meeting Recorded	6,246 words

Metric	Value
Pipeline Task Success Rate	77.27%
Total Flow Runs (past day)	24

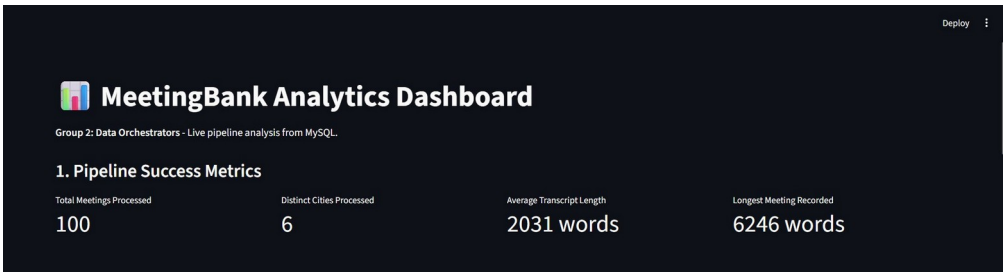

Figure 14: MeetingBank Analytics Dashboard — KPIs confirming 100 total records processed across 6 distinct cities, with an average transcript of 2,031 words.

6.2 Data Volume: Meetings per City

Aggregating by city reveals that Denver City Council (Denvercitycouncil) generated by far the highest volume of meetings (~53 entries) in our dataset, followed by Long Beach (~28), indicating significantly higher governance activity relative to the other four councils.

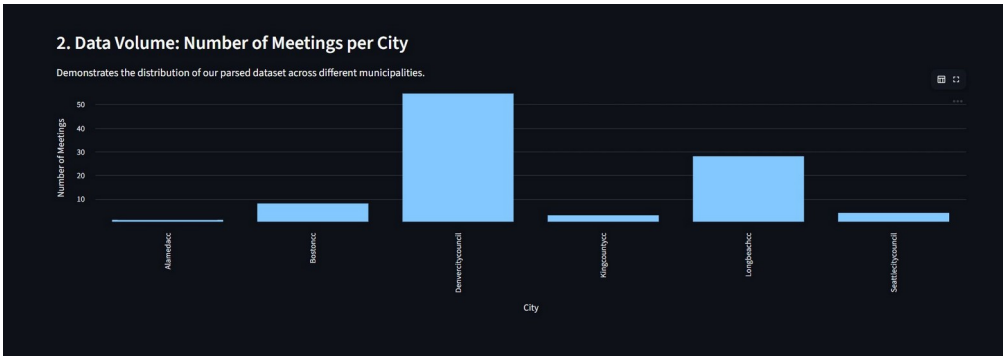

Figure 15: Bar chart showing number of meetings per city council — Denver dominates with ~53 meetings, followed by Long Beach with ~28.

6.3 Content Depth: Average Word Count per City

King County (Kingcountycc) generated the longest average transcripts, exceeding 4,000 words per meeting, highlighting a deeply detailed administrative process compared to other councils. Denver, despite having the most meetings, averages only ~2,300 words per session.

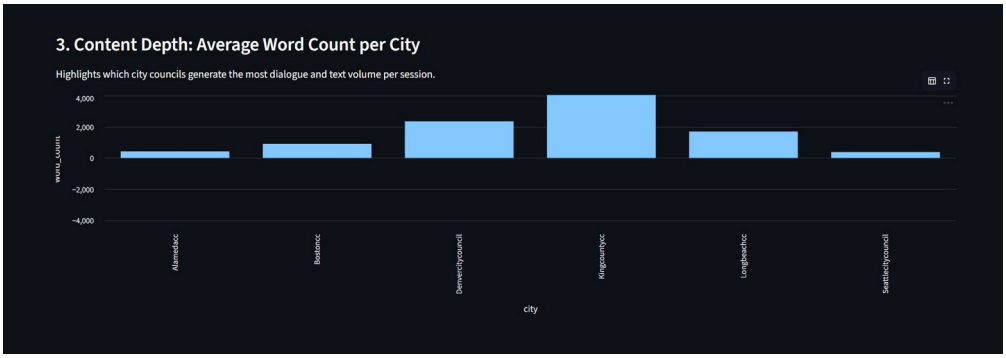

Figure 16: Bar chart showing average word count per city — King County leads with ~4,000 words per meeting, indicating the most detailed deliberations.

6.4 Data Quality: Word Count vs. Character Length

By plotting word_count against char_length for every meeting, we verified a strong linear relationship between the two derived metrics. This perfect linear correlation proves our text extraction worked correctly without corrupting any strings during the ingestion pipeline.

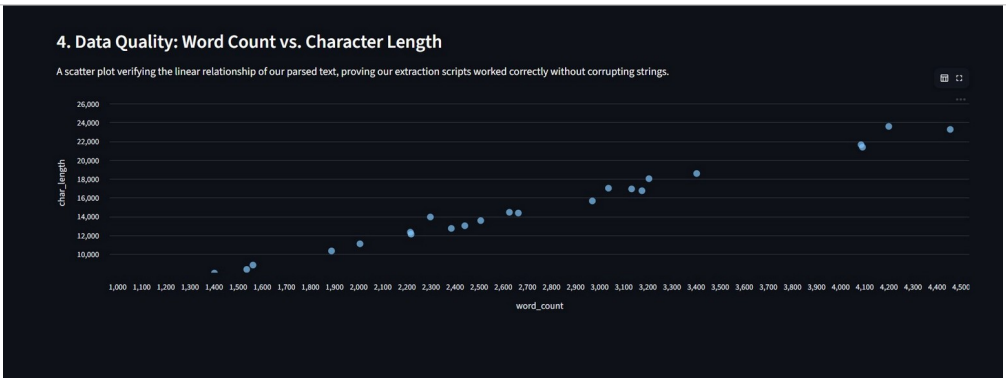

Figure 17: Scatter plot confirming perfect linear correlation between word count and character length — verifying data integrity across all 100 records.

7. Critical Reflection: Challenges and Solutions

A major part of our engineering process involved encountering and overcoming three critical technical hurdles that required us to rethink fundamental aspects of our design.

7.1 Challenge 1: Data Duplication Bug (Idempotency)

The Mistake: During testing, repeated pipeline executions were appending data to the databases rather than replacing it. As a result, our dashboard inaccurately reported 300+ processed meetings when our source file only contained 100.

The Solution: We re-engineered our load tasks to implement Idempotency — ensuring the script can run infinitely without altering the final target state. We safely wipe target tables before inserting fresh data:

```
# The Idempotency Fixes
cursor.execute("TRUNCATE TABLE meetings_metadata;")  # MySQL fix
collection.delete_many({})                           # MongoDB fix
```

7.2 Challenge 2: Database Connection Timeouts

The Mistake: Local MySQL and MongoDB instances occasionally rejected connection requests due to high resource usage, crashing the Python script mid-execution with no recovery path.

The Solution: We utilized Prefect's built-in Fault Tolerance by adding @task(retries=2, retry_delay_seconds=5) to all database loader tasks. The system automatically pauses, waits 5 seconds, and retries — eliminating the need for any manual intervention.

7.3 Challenge 3: Architecture Pivot from Airflow to Prefect

We originally designed our architecture around Apache Airflow. However, during the initial build phase, we found Airflow's setup overhead — including the requirement for a separate metadata database, web server, and scheduler process — too heavy for our specific use case.

We dynamically pivoted to Prefect, which allowed us to define our entire DAG using Python-native @flow and @task decorators within a single file, drastically reducing our technical debt while maintaining enterprise-grade scheduling, retries, and monitoring capabilities.

8. Team Work Allocation & Ethics

8.1 Team Work Allocation (Agile Methodology)

Our team operated using an Agile framework with a Kanban board to track tasks across weekly sprints, assigning work based on individual technical strengths:

Name	Role	Responsibilities
Guna Prakash Kethineni	Lead Data Engineer	Python architecture, Pandas ingestion, Feature Engineering, Streamlit dashboard
Kaustubh Zanan	DB & Orchestration Engineer	MySQL/MongoDB schemas, Prefect DAG deployment code
Manav Chauhan	QA & Analytics Lead	Pipeline testing, Prefect logs, Idempotency validation, Dashboard SQL

Name	Role	Responsibilities
Niranjan V	Project Manager & Scrum Master	Agile sprints, Report formatting, Documentation, Ethics compliance

8.2 Ethical Reflection

While the MeetingBank dataset consists of public city council transcripts, we recognise our ethical responsibilities as data engineers handling governance data:

- Transparency: We maintained clear audit trails ensuring our pipeline's text transformations did not alter the factual context of meeting summaries. Prefect's Event Feed tracked every state change, providing total accountability.
- Public Privacy: Even in public forums, private citizens occasionally speak. We chose to aggregate data at the city and meeting level (averages, word counts) rather than profiling individual speakers, mitigating the risk of deanonymization or misuse of public commentary.

9. Conclusion

Group 2 successfully delivered a robust, end-to-end data engineering pipeline. By overcoming idempotency bugs and network failures, we transitioned a static, unreadable dataset into a dynamic, self-healing, and highly queryable business intelligence tool.

We successfully met all core grading requirements — data ingestion and cleaning, feature engineering and transformation, polyglot storage (MySQL + MongoDB), and querying with live dashboard analytics — while fully realising our Group 2 extension goals through Prefect automation, CRON and interval scheduling, and fault-tolerant DAG orchestration.

This project demonstrates a production-ready pipeline architecture that could be scaled with minor modifications to process the full MeetingBank dataset or any similar governance data corpus.